UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2015-12, page [2]

Unofficial model answer:

a) The inventor referred to as 'Mr. X' is Charles Babbage, who is known as the Father of the Computer. (1 mark)

b) The Y machine is the Difference Engine, which Babbage designed to compute tables of numbers for naval navigation. (1 mark)

c) The Analytical Machine was designed as a general-purpose programmable mechanical computer. Its function was to perform any calculation automatically by following instructions supplied on punched cards, and to produce printed output. It was intended to evaluate mathematical and logical operations, not merely a single fixed task. (1 mark)

d) Mapping the Analytical Machine components to the Von Neumann architecture: the 'memory' (store) corresponds to the main memory; the 'mill' corresponds to the arithmetic logic unit (ALU) and control unit (CPU); the 'punch card reader' corresponds to the input unit; and the 'punch and printed section' corresponds to the output unit. (2 marks)

Marking points:

• *1 mark: (a) Charles Babbage*

• *1 mark: (b) Difference Engine*

• *1 mark: (c) General-purpose programmable computation / automatic calculation using punched-card instructions and printed output*

• *2 marks: (d) Correct mapping of all four components: memory→main memory, mill→ALU/CPU, punch card reader→input, punch/printed section→output (1 mark for partial mapping)*

Model confidence: high.

Model note: The question text contains a typo: 'Name the inventor of Mr. X' is interpreted as asking for the name of Mr. X (Charles Babbage). The Analytical Machine is historically attributed to Charles Babbage (1837). The mapping to Von Neumann architecture is conceptual; the Analytical Machine predates Von Neumann and lacks a stored-program concept in the modern sense, but the component analogy is standard in CSC429.

QUESTION 2 (5 marks)

Source: CSC429-2018-01, page [6]

Unofficial model answer:

a) Classification of parallel processors is constructed according to the level at which parallelism is exploited and the hardware organisation used to exploit it. A common construction (Flynn's taxonomy) classifies processors by the number of instruction streams and data streams: SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data) and MIMD (multiple instruction, multiple data). A broader construction also distinguishes parallel architectures by memory organisation and interconnection: shared-memory multiprocessors (UMA/NUMA), message-passing multicomputers, SIMD array/vector processors, and multithreaded/multicore processors. Thus the classification is built from two dimensions: instruction/data stream multiplicity and the memory/communication model used to connect processing elements.

b) One example for each parallel processor architecture: SISD — a conventional single-core von Neumann CPU (e.g., a basic scalar processor); SIMD — a GPU or vector processor (e.g., CUDA GPU, Intel SSE/AVX vector unit); MISD — a systolic array or fault-tolerant replicated pipeline (rarely used as a general model);

© Hak Cipta Universiti Teknologi MARA

MIMD — a multicore CPU or a cluster of workstations (e.g., Intel Core i7 multicore, Beowulf cluster). If the lecturer's construction uses memory-based classes instead, examples are: shared-memory multiprocessor — multicore CPU with shared L3 cache; message-passing multicomputer — cluster connected by Ethernet/InfiniBand; SIMD array — vector processor; multithreaded — simultaneous multithreading (hyper-threading) core.

Marking points:

• 1 mark: states the classification is based on instruction stream and data stream multiplicity (Flynn's taxonomy) or equivalent dimension

• 1 mark: correctly names/defines SISD, SIMD, MISD, MIMD (or the memory-based classes used in the course)

• 1 mark: gives a valid example for SISD and SIMD

• 1 mark: gives a valid example for MISD and MIMD

• 1 mark: example matches the stated architecture class and uses correct terminology (e.g., vector processor, multicore, cluster)

Model confidence: medium.

Model note: The question does not specify which classification scheme is taught; the answer uses Flynn's taxonomy as the most common construction and also mentions memory-based classification. If the course uses a different scheme (e.g., shared-memory vs message-passing only), the examples should be adjusted accordingly. No figure or table was supplied, so no figure limitation applies.

QUESTION 3 (5 marks)

Source: CSC429-2025-07, page [2]

Unofficial model answer:

a) Inventors: i) Mechanical Calculating Machine — Blaise Pascal (the Pascaline, 1642). ii) Analytical Machine (Analytical Engine) — Charles Babbage (1837). iii) IAS Machine — John von Neumann (Institute for Advanced Study machine, 1945/1952). b) Later-generation machines are general-purpose because they use stored-program architecture (von Neumann model): instructions and data reside in the same rewritable memory, so the same hardware can execute different programs without rewiring. Earlier machines were special-purpose (e.g., Pascal's adder, Babbage's engine, ENIAC's plugboard) with fixed or hard-wired functions. Advances in electronics (transistors, ICs, VLSI) made processors fast, small, cheap and reliable enough to support flexible instruction sets, operating systems and high-level languages, giving versatility, programmability, and better cost-performance across many applications.

Marking points:

• 1 mark: i) Blaise Pascal

• 1 mark: ii) Charles Babbage

• 1 mark: iii) John von Neumann

• 1 mark: General-purpose due to stored-program concept / programmability (instructions and data in same memory, no rewiring)

• 1 mark: Contrast with earlier special-purpose/fixed-function machines and/or role of transistor/IC/VLSI advances enabling flexibility, reliability and cost-effectiveness

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. 'Mechanical Calculating Machine' is interpreted as Pascal's Pascaline; if the paper intended Leibniz's stepped reckoner, accept Gottfried Wilhelm Leibniz. 'Analytical Machine' is Babbage's Analytical Engine. IAS machine is attributed to John von Neumann (with Goldstine and Burks).

QUESTION 4 (5 marks)

Source: CSC429-2025-07, page [2]

Unofficial model answer:

a) A bit (binary digit) is the smallest unit of information in a computer, representing exactly one of two states, conventionally 0 or 1. Physically it can be realised by any two-state device (a vacuum tube ON/OFF, a transistor, a magnetic cell, a capacitor charge). It is the basic building block from which larger units are formed: 1 nibble = 4 bits, 1 byte = 8 bits, 1 word = typically 16/32/64 bits.

b) Workflow: (1) interpret 'gro' as the decimal number 90 (g=9, r=0 in the usual base-10 reading; this is the only reading that makes the question well-posed). (2) Convert 90 into the required base. (3) Count the digits in that representation, since each digit is stored in one vacuum tube (one tube = one bit-position/state).

i) Binary system: $90 = 64 + 16 + 8 + 2 = 1011010_2$. This representation has 7 binary digits, so 7 vacuum tubes are needed (each tube holds one bit, ON=1/OFF=0). Check: 2^6 = 64 ≤ 90 < 128 = 2^7, so $\lceil \log_2(91) \rceil = 7$ bits.

ii) Octal system: $90 = 1\times64 + 3\times8 + 2 = 132_8$. In an octal system each octal digit is itself represented by 3 bits (a group of 3 tubes), so the number of tubes = 3 digits × 3 bits/digit = 9 tubes. Equivalently $132_8 = 001\ 011\ 010_2 = 9$ bits. (If the convention is that one tube stores one octal digit directly, the count would be 3 tubes; the standard bit-per-tube interpretation gives 9.)

Marking points:

• 1 mark (a): defines a bit as the smallest two-state unit of information (0/1), correctly identifying it as a binary digit.

• 1 mark (b): states the workflow — read 'gro' as decimal 90, convert to the target base, then count digits/bits per tube.

• 1 mark (b)(i): correct binary conversion $90 = 1011010_2$.

• 1 mark (b)(i): concludes 7 vacuum tubes (one tube per bit).

• 1 mark (b)(ii): correct octal conversion $90 = 132_8$ and concludes 9 tubes (3 bits per octal digit), with working shown.

Model confidence: medium.

Model note: The question text is garbled: '\Mat is a bit?' is read as 'What is a bit?', and 'gro' is assumed to mean the decimal value 90 (digits g=9, r=0). If 'gro' was intended as a different literal or a base-notation value, the tube counts change accordingly. Also, part (b)(ii) is ambiguous: if one tube is taken to store one octal digit, the answer is 3 tubes; the standard one-bit-per-tube interpretation gives 9. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 5 (10 marks)

Source: CSC429-2019-06, page [5]

Unofficial model answer:

a) Two functions of an I/O module: (1) Interface to the CPU and memory: the I/O module provides the necessary control and data buffering to transfer data between the system bus and external devices, handling address decoding, synchronization, and status reporting. (2) Device control and data transfer: it controls the operation of the peripheral device, performs data conversion (e.g., serial-to-parallel), and manages the transfer of data to/from the device, often using interrupts or DMA to notify the CPU of completion. (Other acceptable functions: error detection, buffering, etc.)

b) Two categories of external I/O devices with examples: (1) Human-readable: devices that communicate with the user, e.g., keyboard, monitor, printer. (2) Machine-readable: devices that communicate with equipment, e.g., disk drives, sensors, actuators. (The third category is communication devices, e.g., modem, network interface.)

c) I/O technique requiring active CPU intervention: programmed I/O (or interrupt-driven I/O, where CPU executes ISR). I/O technique not requiring CPU intervention: direct memory access (DMA).

Marking points:

• 1 mark: Correctly states first function of I/O module (e.g., CPU/memory interface, buffering, control).

• 1 mark: Correctly states second function of I/O module (e.g., device control, data conversion, error detection).

• 1 mark: Lists one correct category of external I/O devices (human-readable, machine-readable, or communication).

• 1 mark: Provides appropriate example for that category.

• 1 mark: Lists second correct category of external I/O devices.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Provides appropriate example for that category.

• 1 mark: Names one I/O technique requiring active CPU intervention (programmed I/O or interrupt-driven I/O).

• 1 mark: Names one I/O technique not requiring CPU intervention (DMA).

• 1 mark: Explanation/accuracy of functions (additional clarity).

• 1 mark: Overall correctness and completeness (e.g., no contradictions, proper terminology).

Model confidence: high.

Model note: The question does not specify the exact categorization scheme; the answer uses the common human-readable, machine-readable, and communication categories. Other categorizations (e.g., by data transfer direction) may be acceptable if justified. The marking scheme is indicative and not official.

QUESTION 6 (10 marks)

Source: CSC429-2025-07, page [3]

Unofficial model answer:

Part (a) — Access methods (3 marks). The scenarios map to the three classic file/memory access methods: sequential, direct (random), and indexed. (i) *Read a C file from a folder* — this is **direct (random) access**: the file is located and opened by its name/path without reading preceding files, so the system jumps straight to the required block. (ii) *Access a file line by line / streaming a video frame by frame* — this is **sequential access**: records are read in order, one after another, from start to end (a stream must be consumed in sequence). (iii) *Access a specific file block using an index in a large file* — this is **indexed access**: an index table is searched to find the pointer to the desired block, which is then accessed directly. (iv) *Jump to a specific frame or timestamp in a video* — this is **direct/random access** (often supported by an index of timestamps), since the player seeks to an arbitrary position rather than scanning from the beginning. [Note: the extracted question text is garbled; the mapping above follows the standard CSC429 access-method taxonomy — sequential, direct/random, indexed — and the visible scenario keywords 'line by line', 'index', 'specific frame/timestamp'.]

Part (b) — Ranking by storage capacity (4 marks). From most to least capacity: **Tape > Hard drive > Main memory > L3 Cache > L1 Cache > Register.** Tape (magnetic tape) offers the largest capacity at lowest cost per bit; hard disk is next; main memory (RAM) is smaller than secondary storage; cache levels shrink as they get closer to the CPU (L3 larger than L1); registers are the smallest storage inside the CPU. This ordering also reflects the memory hierarchy: capacity increases as you move away from the processor while speed and cost per bit decrease.

Part (c) — Volatile memory (3 marks). *Volatile* means the memory **loses its stored contents when power is removed or interrupted**. Volatile memory (e.g., registers, cache, main memory/DRAM and SRAM) requires a continuous power supply to retain data, so its contents are temporary and must be reloaded from non-volatile secondary storage after a power cycle or reboot. Volatile memory is typically faster and is used to hold the currently executing program and data, whereas non-volatile memory (hard disk, SSD, ROM, tape) retains data without power and is used for permanent storage.

Marking points:

• 1 mark: (a) correctly identifies direct/random access for reading a named C file from a folder (with brief justification).

• 1 mark: (a) correctly identifies sequential access for reading a file line by line / streaming video in order.

• 1 mark: (a) correctly identifies indexed access for accessing a specific file block via an index, and direct/random access for jumping to a specific video frame/timestamp.

• 1 mark: (b) Tape ranked first (largest capacity).

• 1 mark: (b) Hard drive ranked second, above main memory.

• 1 mark: (b) Main memory > L3 Cache > L1 Cache correctly ordered.

• 1 mark: (b) Register ranked last (smallest) and full ordering correct: Tape > Hard drive > Main memory > L3 > L1 > Register.

• 1 mark: (c) defines volatile as losing contents when power is removed/interrupted.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: (c) gives examples of volatile memory (registers, cache, RAM/DRAM/SRAM) and states it needs continuous power.

• 1 mark: (c) contrasts with non-volatile memory (disk/SSD/ROM/tape) which retains data without power, and/or notes data must be reloaded after power loss.

Model confidence: medium.

Model note: The question text for part (a) is badly garbled in the extracted paper (OCR errors: 'Readi l files', 'Streami files line line', 'AS ecific file block usi an index', 'toas cific frame or a timestam'). The mapping of scenarios to access methods is inferred from the visible keywords and the standard CSC429 sequential/direct/indexed taxonomy; exact scenario wording could not be verified. Part (b) ranking assumes standard capacity ordering of the memory hierarchy. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2025-02, page [3]

Unofficial model answer:

(a) Convert −81.21875 to IEEE 754 single precision (hex). Step 1 — sign: the number is negative, so S = 1. Step 2 — magnitude in binary: 81 = 64 + 16 + 1 = 1010001_2, and 0.21875 = 0.21875 × 2 = 0.4375 (0), ×2 = 0.875 (0), ×2 = 1.75 (1), ×2 = 1.5 (1), ×2 = 1.0 (1) → 0.00111_2. So 81.21875 = 1010001.00111_2. Step 3 — normalise: $1010001.00111_2 = 1.01000100111_2 \times 2^6$, so the exponent field E = 6 + 127 = 133 = 10000101_2 and the fraction (mantissa) is 01000100111 followed by zeros to 23 bits: 01000100111000000000000. Step 4 — assemble: S | E | M = 1 10000101 01000100111000000000000. Grouping into nibbles: 1100 0010 1010 0010 0111 0000 0000 0000 = C2A27000h. Answer: C2A27000h. (b) Convert 43A37C00h to decimal. Step 1 — expand to binary: 4=0100, 3=0011, A=1010, 3=0011, 7=0111, C=1100, 0=0000, 0=0000 → 0100 0011 1010 0011 0111 1100 0000 0000. Step 2 — split fields: S = 0 (positive), E = 10000111_2 = 135, M = $01000110111110000000000_2$. Step 3 — unbiased exponent: 135 − 127 = 8. Step 4 — significand: 1.M = 1.0100011011111_2. Step 5 — value = $1.0100011011111_2 \times 2^8 = 101000110.11111_2$ (shift the point 8 places right). Step 6 — convert: integer part 101000110_2 = 256 + 64 + 4 + 2 = 326; fraction 0.11111_2 = 1/2 + 1/4 + 1/8 + 1/16 + 1/32 = 31/32 = 0.96875. Total = 326.96875. Answer: +326.96875.

Marking points:

• (a) 1 mark: correct sign bit S = 1 for the negative number

• (a) 1 mark: correct binary of magnitude 81.21875 = 1010001.00111_2 (integer and fractional parts)

• (a) 1 mark: correct normalisation $1.01000100111_2 \times 2^6$ and biased exponent 133 = 10000101_2

• (a) 1 mark: correct 23-bit mantissa 01000100111000000000000

• (a) 1 mark: correct final hex C2A27000h (consistent with fields)

• (b) 1 mark: correct binary expansion of 43A37C00h and field split S=0, E=10000111_2, M=0100011011111…

• (b) 1 mark: correct unbiased exponent 135 − 127 = 8

• (b) 1 mark: correct significand 1.0100011011111_2 and re-placement of binary point (101000110.11111_2)

• (b) 1 mark: correct integer part 326

• (b) 1 mark: correct fraction 0.96875 and final answer +326.96875

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. No figure/table was supplied; all values derived from the question text. Marks assume an even 5+5 split across subparts (a) and (b).

QUESTION 8 (10 marks)

Source: CSC429-2026-02, page [3, 4]

Unofficial model answer:

a) Definition: An interrupt is a signal (hardware or software) sent to the CPU that causes it to suspend the currently executing program, save its state (context), and transfer control to a dedicated interrupt service

routine (ISR) or interrupt handler; after the handler completes, the saved context is restored and the interrupted program resumes. Interrupts allow the CPU to respond to asynchronous external events (I/O completion, timers, errors) and synchronous traps without polling.

b) Highest priority: Interrupt W. In a nested-interrupt timing diagram, the handler that is entered first and is never preempted by any other handler has the highest priority. W begins first and runs to completion while X, Y and Z are all shown nested inside/after it, so W preempts all others and therefore has the highest priority.

c) Lowest priority: Interrupt Z. Z is the handler that is entered last and is nested deepest (it is the innermost, most recently started handler). A handler that can be preempted by every other handler and only runs when nothing higher is pending has the lowest priority, which matches Z being the last/deepest nested handler in Figure 2.

d) Priority (nested) completion sequence: Because a higher-priority interrupt preempts a lower-priority one, the handler that starts last (lowest priority) finishes first and the highest-priority handler finishes last. The completion order is therefore Z, then Y, then X, then W (i.e., Z → Y → X → W). Equivalently, the service/entry order is W → X → Y → Z and completion is the reverse.

e) Disable-nested (non-nested) completion sequence: When nested interrupts are disabled, once a handler begins, all further interrupts are masked until that handler finishes; pending interrupts are then serviced one at a time in priority order. Hence each handler runs to completion before the next is served, giving completion order W, then X, then Y, then Z (W → X → Y → Z), i.e., the same order in which they arrived/are prioritized.

Marking points:

• *1 mark (a): Correct definition of interrupt — signal to CPU causing suspension of current program, saving context, and transfer to an ISR, with resumption afterwards.*

• *2 marks (b): Identifies W as highest priority (1 mark) with justification that it is entered first / never preempted / outermost handler (1 mark).*

• *2 marks (c): Identifies Z as lowest priority (1 mark) with justification that it is entered last / deepest nested / preempted by all others (1 mark).*

• *2 marks (d): Correct completion sequence Z → Y → X → W (2 marks); 1 mark if order reversed or partially correct.*

• *2 marks (e): Correct completion sequence W → X → Y → Z (2 marks); 1 mark if order reversed or partially correct.*

• *1 mark: Consistent use of correct terminology (preemption, nesting, priority, ISR/context save) throughout.*

Model confidence: medium.

Model note: Figure 2 was supplied only as extracted text/caption ('Nested interrupts') without a clear timing diagram; the answer assumes the standard nested-interrupt timing diagram in which W is entered first and is never preempted (highest priority) and Z is entered last and nested deepest (lowest priority). If the actual figure shows a different entry/preemption pattern, the priority ranking and completion sequences in (b)-(e) would change accordingly. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2026-02, page [5]

Unofficial model answer:

a) An I/O module is installed when a computer system needs to connect to and communicate with external devices (peripherals) such as keyboards, displays, disks, or network adapters. It is required whenever the CPU must transfer data to or from a device that is not directly connected to the system bus, providing the interface, control, and data buffering needed for the exchange.

b) If the I/O module is not installed, the CPU cannot communicate with external devices. The system would be unable to perform input or output operations, meaning no data could be read from or written to peripherals; the computer would be isolated and unable to interact with the outside world, severely limiting its usefulness.

c) One category of I/O module is the 'human-readable' or 'user-interface' module. This category includes devices that allow humans to interact with the computer, such as keyboards, mice, monitors, and printers.

© Hak Cipta Universiti Teknologi MARA

These modules convert between the computer's internal binary representation and forms that humans can perceive or manipulate (e.g., text on a screen, printed characters, or keystrokes).

d) 'Hot connection' (hot plugging) refers to the ability to connect or disconnect a device to or from a computer system while the system is powered on and running, without causing damage or requiring a reboot. The system automatically detects and configures the device, allowing immediate use.

e) Three common interfaces to external devices are: (1) USB (Universal Serial Bus), (2) HDMI (High-Definition Multimedia Interface), and (3) Ethernet (for network connections). Other acceptable examples include VGA, Thunderbolt, SATA, and Bluetooth.

Marking points:

• 1 mark: (a) Correctly states that an I/O module is installed when the system needs to connect to external devices/peripherals.

• 1 mark: (b) Explains that without the I/O module, the CPU cannot communicate with external devices, so input/output operations are impossible.

• 2 marks: (c) Identifies one category of I/O module (e.g., human-readable, machine-readable, communication) and explains its function with examples.

• 2 marks: (d) Defines 'hot connection' as connecting/disconnecting a device while the system is powered on, with automatic detection/configuration.

• 3 marks: (e) Lists three correct common interfaces to external devices (1 mark each).

• 1 mark: Overall clarity and correct use of terminology.

Model confidence: high.

Model note: The question does not refer to any figure or table. The answer assumes standard computer organization definitions. For part (c), any valid category is acceptable; the example given is human-readable. For part (e), many interfaces are valid; the marking should accept any three correct ones.

QUESTION 10 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) The main function of cache memory is to reduce the average time to access main memory by holding frequently accessed instructions and data in a small, fast memory close to the CPU. It exploits locality of reference (temporal and spatial) so that most CPU requests are satisfied from cache rather than slower main memory, thereby bridging the speed gap between the processor and main memory and improving overall system performance.

b) Data in cache and main memory are synchronized by using a write policy. In write-through, every write to cache is also written immediately to main memory, keeping them consistent at the cost of more memory traffic. In write-back, writes update only the cache block and set a dirty bit; the block is written back to main memory only when it is replaced, reducing traffic but requiring a dirty-bit mechanism. Cache coherence protocols (e.g., snooping or directory-based) are also used in multiprocessor systems to keep multiple caches and main memory consistent.

c) SRAM vs DRAM: (i) Type of storage: SRAM stores each bit in a bistable flip-flop (static), while DRAM stores each bit as charge on a capacitor (dynamic). (ii) Availability of charge leak: SRAM has no charge leakage because it uses active transistors holding state, whereas DRAM suffers charge leakage from capacitors and must be refreshed periodically. (iii) Price: SRAM is more expensive per bit (larger cell area, more transistors), while DRAM is cheaper per bit (smaller cell, one transistor plus capacitor). (iv) Example of location of memory usage: SRAM is used in CPU cache and registers; DRAM is used for main memory (RAM modules).

Marking points:

• 1 mark: cache reduces average memory access time / bridges CPU-memory speed gap

• 1 mark: exploits locality of reference (temporal/spatial)

© Hak Cipta Universiti Teknologi MARA

- *1 mark: write-through policy keeps cache and main memory consistent*
- *1 mark: write-back policy with dirty bit writes back on replacement*
- *1 mark: cache coherence protocols for multiprocessor synchronization*
- *1 mark: SRAM type of storage = bistable flip-flop / static*
- *1 mark: DRAM type of storage = capacitor charge / dynamic*
- *1 mark: SRAM no charge leakage vs DRAM charge leakage and refresh*
- *1 mark: SRAM more expensive per bit vs DRAM cheaper per bit*
- *1 mark: SRAM used in cache/registers vs DRAM used in main memory*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied beyond the question text, so the answer is based on standard computer-organization concepts.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2026-02, page [6]

Unofficial model answer:

Interpretation of Figure 5: The expression is read as z = A + (C²/B)·D − 3A, i.e. z = A + ((C*C)/B)*D − 3*A. Registers A, B, C, D are fixed (read-only operands); x, y, z are scratch; the final result must end in z. Note: the OCR text 'z-A+c2/B*D-3A' is ambiguous, so the expression above is the assumed reading (see notes).

(a) Zero-address (stack) program. All operands must be pushed onto the stack; results are left on the stack and finally popped into z. Using the stack, compute C*C, divide by B, multiply by D, then add A and subtract 3*A:

```
PUSH C
PUSH C
MUL
PUSH B
DIV
PUSH D
MUL
PUSH A
ADD
PUSH A
PUSH A
ADD
PUSH A
ADD
SUB
POP z
```

(Equivalently, the 3A term can be formed as PUSH A / PUSH A / ADD / PUSH A / ADD, then SUB.) The stack holds the running value; the last POP z stores the final answer in z.

(b) One-address program. One operand is the accumulator (implicit) and the other is a named register; results go to the accumulator. Use x and y as temporaries:

```
LOAD C
MUL C
STOR x
LOAD x
DIV B
STOR x
LOAD x
MUL D
STOR x
LOAD A
ADD x
STOR y
LOAD A
MUL A
ADD A
ADD A
```

© Hak Cipta Universiti Teknologi MARA

STOR x
LOAD y
SUB x
STOR z

Here x holds the C²/B·D term, y holds A + that term, and x is reused to hold 3A; the final SUB leaves the answer in the accumulator, which is stored to z.

(c) Three-address program. Each instruction names two sources and one destination, so the expression is built directly with temporaries x and y:

MUL (C, C, x)
DIV (x, B, x)
MUL (x, D, x)
ADD (A, x, y)
MUL (A, A, x)
ADD (x, A, x)
ADD (x, A, x)
SUB (y, x, z)

This computes x = C²/B·D, y = A + x, x = 3A, and finally z = y − x = A + C²/B·D − 3A, with the answer in z as required.

Marking points:

• *2 marks: correct interpretation of Figure 5 as z = A + (C*C)/B*D - 3*A (or clearly stated equivalent)*

• *6 marks: zero-address program — correct PUSH/POP usage, correct operator order (MUL, DIV, MUL, ADD, SUB), final POP z (2 marks per logical stage)*

• *6 marks: one-address program — correct LOAD/STOR with accumulator, correct use of temporaries x and y, correct final STOR z*

• *4 marks: three-address program — correct ADD/SUB/MUL/DIV (p,q,r) form, correct temporaries, final result in z*

• *2 marks: overall correctness — final answer stored in z in all three versions and fixed registers A, B, C, D never overwritten*

Model confidence: medium.

*Model note: Figure 5 is supplied only as OCR text 'z-A+c2/B*D-3A'; the expression was interpreted as z = A + C^2/B*D - 3A. If the intended expression differs (e.g. z = A + C^2/(B*D) - 3A or z = A + C^2/B*D - 3*A with different grouping), the instruction sequences must be re-ordered accordingly. Table 1 gives only the instruction mnemonics; operand conventions (e.g. whether one-address ADD p means ACC <- ACC + p) are assumed standard. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA